

Finite-Field Edge Inference with GF(137) Kernels and Compact Virtual Quantum Gates

I. Mikhailov

May 2026

Abstract

This paper describes an experimental finite-field compute stack for edge inference. Dense neural-network operations are moved from float32 weights to compact unsigned-byte residues in $\mathbb{Z}/137\mathbb{Z}$, and the matrix product, modular reduction, and threshold activation are fused in native C++. The same finite-field representation is also used to construct a compact virtual quantum processor (VQP) toy model. The reported results are engineering benchmarks, not claims of artificial general intelligence or physical quantum advantage.

1 Architecture

The repository separates the stack into three layers:

$$\text{C}++ \text{ Core} \rightarrow \text{Python ctypes Bindings} \rightarrow \text{Application Modules.} \quad (1)$$

The application layer contains observational cosmology, particle-formula bookkeeping, experimental cryptography, biology-inspired redundancy tests, and virtual quantum gates. The high-performance path is the C++ core:

- `src/modular_core/fused_matrix_gf137.cpp` for fused GF(137) inference;
- `src/modular_crypto/e5137_protocol.cpp` for redundancy and toy ciphers;
- `src/modular_biology/dna_fault_tolerance.cpp` for fault-tolerance tests;
- `src/modular_quantum/vqp_core.cpp` for compact virtual quantum gates.

2 GF(137) Neural Inference

The modular network stores weights and activations as residues

$$x, w, b \in \mathbb{Z}/137\mathbb{Z}. \quad (2)$$

A linear layer is evaluated as

$$y = (xW + b) \bmod 137, \quad (3)$$

followed by the threshold activation

$$a_i = \begin{cases} 1, & y_i \geq 42, \\ 0, & y_i < 42. \end{cases} \quad (4)$$

The fused C++ kernel combines the matrix product, Barrett reduction modulo 137, and threshold activation in a cache-local pass. On ARM64, the optimized shape uses NEON SIMD lanes for hidden accumulation and active output-weight summation. A native repeated-call path measures throughput without paying the Python/ctypes boundary on every forward pass.

3 Edge-AI Benchmark

The benchmark task is a deliberately hard prime/composite classification test using the first 50 fractional base- e digits as features. The modular model is not more accurate than the baseline on this task; the useful result is the memory and runtime profile.

Table 1: Selected final shootout rows. Times are for 1000 forward passes.

Backend	Memory KB	Time ms	Notes
float32 NumPy MLP	6.50391	58.7784	baseline
GF(137) compact uint8	1.62793	582.904	minimal storage, Python overhead
GF(137) C++ fused uint8	1.62793	40.1530	Python-call latency path
GF(137) C++ NEON native loop	1.62793	12.1055	throughput path
JAX JIT CPU	6.50391	42.4758	float compute cache

The C++ NEON throughput row keeps the compact `uint8` representation and is approximately

$$\frac{6.50391}{1.62793} \simeq 4 \quad (5)$$

times smaller than the float32 baseline. It is also about

$$\frac{58.7784}{12.1055} \simeq 4.86 \quad (6)$$

times faster than the NumPy float32 baseline in the recorded throughput mode.

4 Virtual Quantum Processor

The virtual quantum processor is a finite-field gate emulator, not a physical quantum simulator. A system of n virtual qubits is represented by

$$nD = 26n \quad (7)$$

bytes of phase-lane state rather than by 2^n complex amplitudes. The VQP therefore has linear memory complexity

$$O(nD), \quad (8)$$

while a conventional dense state vector has memory complexity

$$O(2^n). \quad (9)$$

In the toy semantics, the Hadamard-like gate is a finite-field phase mixing using the inverse of 2 modulo 137:

$$2^{-1} \equiv 69 \pmod{137}. \quad (10)$$

The CNOT-like gate hashes two virtual qubits across the 26 axes and applies the target update if the interaction residue is at least 42.

5 VQP Benchmark

The Phase 8 VQP row already showed a compact-state advantage. The Phase 9 optimization exploits an exact property of the toy semantics: the target-oracle projection overwrites all phase lanes, making repeated Grover-to-target calls idempotent. The repeated path can therefore collapse to a single projected basis-state write. On ARM builds, that projected write uses guarded NEON stores.

Table 2: VQP compact benchmark against a local NumPy complex128 state-vector baseline.

Qubits	Repeats	VQP ms	Float ms	Speedup	Memory ratio
8	5000	0.072708	45.932166	631.73×	19.69×
10	3000	0.006709	49.136333	7323.96×	63.02×
12	1000	0.012459	37.761125	3030.83×	210.05×
16	300	0.019291	94.975167	4923.29×	2520.62×

The earlier Phase 8 non-collapsed benchmark gave a 16-qubit speedup of roughly 95×. The Phase 9 row is much faster because the repeated benchmark path uses algebraic idempotence of the toy gate semantics. This is a representation optimization, not a claim of quantum advantage over a general state-vector simulator.

6 Testing and Reproducibility

The active test suite is intentionally compressed to five super-tests:

1. field and lepton core;
2. hadron phase gate;
3. cosmological vacuum limits;
4. topological fault tolerance;
5. dark-matter and gamma phenomenology.

The current release is verified by

```
uv run pytest
```

which returns

```
5 passed
```

7 Conclusion

The GF(137) stack is useful as a small edge-compute experiment: it demonstrates compact byte-level storage, fused modular inference, and native C++/NEON throughput advantages on a constrained benchmark. The VQP layer extends the same representation to gate-like finite-field transitions. The results should be interpreted as systems-engineering artifacts rather than as AGI, cryptographic security, or physical quantum-computing claims.